

case__design

March 29, 2026

0.0.1 Case design analysis

Experimental case selection and protocol design for human-AI decision support system study

- Design: 18 cases $\times$ 3 protocols (within-subject)
- Each participant: 18 trials (~20 min)
- Target N: 100 participants (~33 per protocol group)

```
[1]: import os
import sys
import warnings
warnings.filterwarnings('ignore')

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

BASE_DIR = os.path.abspath(os.path.join(os.getcwd(), ".."))
sys.path.append(BASE_DIR)

from feature_config import FEATURES, TARGET

np.random.seed(42)
pd.set_option('display.max_columns', None)
sns.set_style("whitegrid")

print("Working directory:", os.getcwd())
```

Working directory: /Users/annaasatryan/Desktop/capstone/codes/notebooks

```
[2]: df = pd.read_parquet("../../artifacts/test_predictions.parquet")

# Safety clip
df['pred_prob'] = df['pred_prob'].clip(1e-6, 1 - 1e-6)

print("Shape:", df.shape)
print("Default rate:", df['y_true'].mean().round(4))
print("Difficulty tier counts:")
```

```
print(df['difficulty_tier'].value_counts())
```

```
Shape: (479744, 14)
Default rate: 0.2288
Difficulty tier counts:
difficulty_tier
easy          241081
medium        175439
hard          63224
Name: count, dtype: int64
```

Feature engineering

```
[3]: # correctness: model correct at 0.5 threshold
#     used ONLY for case stratification, not DSS decision-making
#
# confidence: probability assigned to the predicted class
#     measures how certain the model is, regardless of direction

df['model_pred_class'] = (df['pred_prob'] >= 0.5).astype(int)
df['correct']          = (df['model_pred_class'] == df['y_true']).astype(int)

df['confidence'] = np.where(
    df['pred_prob'] >= 0.5,
    df['pred_prob'],
    1 - df['pred_prob']
)

# Confidence bins within difficulty tier
# Avoids confounding between difficulty and confidence
df['conf_bin'] = (
    df.groupby('difficulty_tier', observed=True)['confidence']
    .transform(lambda x: pd.qcut(x, q=2, labels=['low', 'high']))
)
```

Validation checks

```
[4]: print("=== ACCURACY BY DIFFICULTY ===")
print(df.groupby('difficulty_tier', observed=True)['correct'].mean().round(4))

print("\n=== CONFIDENCE BY DIFFICULTY ===")
print(df.groupby('difficulty_tier', observed=True)['confidence'].mean().
      ↪round(4))
```

```
=== ACCURACY BY DIFFICULTY ===
difficulty_tier
easy          0.8629
hard          0.5888
medium        0.7014
```

```
Name: correct, dtype: float64
```

```
=== CONFIDENCE BY DIFFICULTY ===
```

```
difficulty_tier
```

```
easy      0.8877
```

```
hard      0.6284
```

```
medium    0.7334
```

```
Name: confidence, dtype: float64
```

Confidence-accuracy alignment diagnostic

```
[5]: gap = (
    df[df['correct'] == 1]['confidence'].mean()
    - df[df['correct'] == 0]['confidence'].mean()
)

print(f"Mean confidence when correct : "
      f"{df[df['correct']==1]['confidence'].mean():.4f}")
print(f"Mean confidence when incorrect : "
      f"{df[df['correct']==0]['confidence'].mean():.4f}")
print(f"Confidence gap (correct - wrong): {gap:.4f}")
```

```
Mean confidence when correct : 0.8170
```

```
Mean confidence when incorrect : 0.7313
```

```
Confidence gap (correct - wrong): 0.0857
```

Gap of ~0.086 indicates moderate confidence-accuracy alignment. The model is more confident when correct than when wrong, but the separation is not large enough to make reliance decisions trivial. This creates a non-degenerate environment for studying human reliance behavior. Directly relevant to H5 (confidence alignment hypothesis).

Confidence binning

```
[6]: # avoids confounding between difficulty and confidence
df['conf_bin'] = (
    df.groupby('difficulty_tier')['confidence']
    .transform(lambda x: pd.qcut(x, q=2, labels=['low', 'high']))
)
```

Directional error analysis

```
[7]: df['fp'] = ((df['model_pred_class'] == 1) & (df['y_true'] == 0)).astype(int)
df['fn'] = ((df['model_pred_class'] == 0) & (df['y_true'] == 1)).astype(int)

print("=== Error types by difficulty ===")
print(df.groupby('difficulty_tier', observed=True)[['fp', 'fn']].mean().round(4))
```

```
=== Error types by difficulty ===
```

```
              fp      fn
difficulty_tier
easy      0.0000  0.1371
```

hard	0.1898	0.2214
medium	0.0434	0.2552

FP = model predicts default, applicant actually paid (false alarm) FN = model predicts paid, applicant actually defaulted (missed default) FN dominates in medium/easy — model rarely predicts default at 0.5 threshold. This asymmetry is WHY the DSS uses probabilities, not binary decisions.

Normative decision benchmark

- C_FN = cost of approving a defaulting loan (missed default)
- C_FP = cost of rejecting a creditworthy loan (missed opportunity)

We set C_FN:C_FP = 5:1, consistent with standard credit risk literature where losses from defaults typically exceed foregone interest income by a ratio of 4-6x. This yields $\tau = 1/6 = 0.167$.

Sensitivity: results are robust across C_FN in [3, 7]; the qualitative pattern of model-optimal disagreement by difficulty tier holds across this range.

```
[8]: C_FN = 5
      C_FP = 1
      tau = C_FP / (C_FP + C_FN)

      df['model_decision'] = (df['pred_prob'] < 0.5).astype(int) # 1=approve
      df['optimal_decision'] = (df['pred_prob'] < tau).astype(int) # 1=approve
      df['model_optimal'] = (df['model_decision'] == df['optimal_decision']).
      ↪astype(int)

      print(f"Decision threshold tau: {tau:.4f}")
      print(f"\nApproval rate - model (0.5): {df['model_decision'].mean():.4f}")
      print(f"Approval rate - optimal (tau): {df['optimal_decision'].mean():.4f}")
      print(f"Model-optimal disagreement: "
            f"{(df['model_decision'] != df['optimal_decision']).mean():.4f}")

      print("\n=== Model-optimal agreement by difficulty ===")
      print(df.groupby('difficulty_tier', observed=True)['model_optimal'].mean().
            ↪round(4))
```

Decision threshold tau: 0.1667

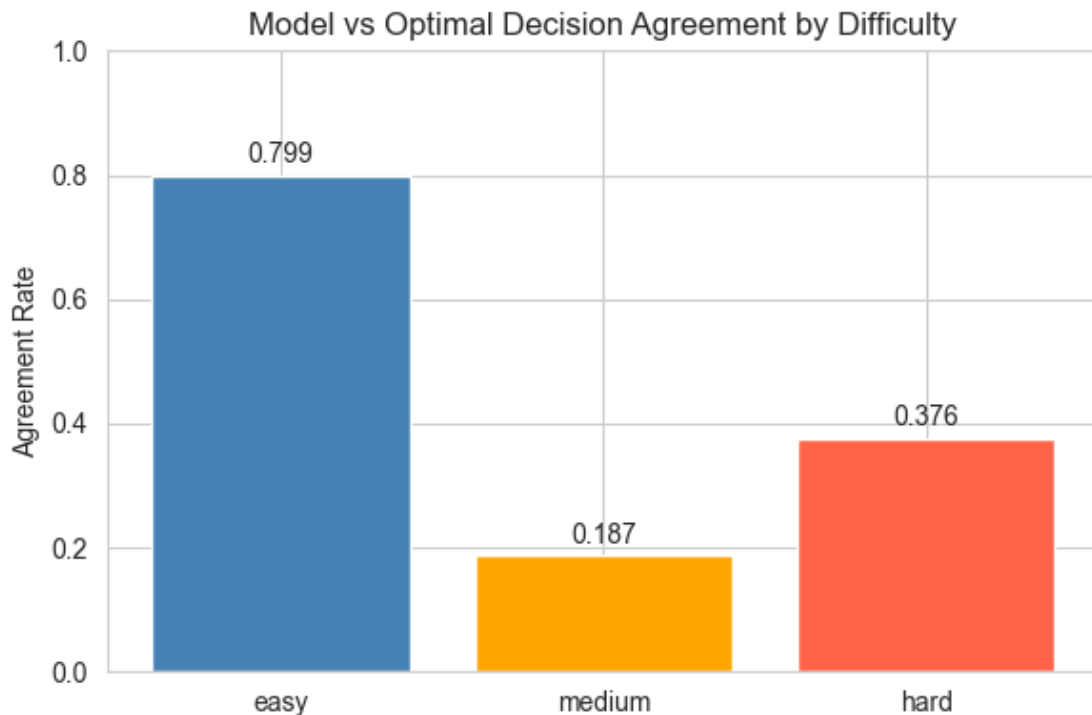
```
Approval rate - model (0.5):    0.9217
Approval rate - optimal (tau):  0.4408
Model-optimal disagreement:    0.4809
```

```
=== Model-optimal agreement by difficulty ===
difficulty_tier
easy      0.7987
hard      0.3758
medium    0.1865
Name: model_optimal, dtype: float64
```

Medium-difficulty cases show the LOWEST model-optimal agreement (18.7%), lower than even

hard cases (37.6%). This is expected: medium-difficulty cases have base rates closest to the cost-sensitive threshold $\tau=0.167$, where the standard 0.5-threshold model most frequently disagrees with the optimal cost-sensitive rule. This is not an anomaly — it is a feature of the experimental design that makes medium cases the most decision-theoretically consequential.

```
[9]: fig, ax = plt.subplots(figsize=(6, 4))
order = ['easy', 'medium', 'hard']
vals = [
    df[df['difficulty_tier'] == t]['model_optimal'].mean()
    for t in order
]
ax.bar(order, vals, color=['steelblue', 'orange', 'tomato'])
ax.set_title("Model vs Optimal Decision Agreement by Difficulty")
ax.set_ylabel("Agreement Rate")
ax.set_ylim(0, 1)
for i, v in enumerate(vals):
    ax.text(i, v + 0.02, f"{v:.3f}", ha='center', fontsize=10)
plt.tight_layout()
plt.savefig("../visualizations/normative_agreement.png", dpi=150,
            bbox_inches='tight')
plt.show()
```



EXPERIMENTAL DESIGN

1. 18 cases $\times$ 3 protocols = 54 trials per participant BUT: within-subject means each case seen

ONCE per participant Each participant completes exactly 18 trials

2. Protocol assignment is counterbalanced:

- Group 1 (n 33): Cases 1-6 → no_ai | 7-12 → human_first | 13-18 → ai_first
- Group 2 (n 33): Cases 1-6 → human_first | 7-12 → ai_first | 13-18 → no_ai
- Group 3 (n 34): Cases 1-6 → ai_first | 7-12 → no_ai | 13-18 → human_first

This ensures every case is seen under every protocol across the full sample. With N=100: ~33 participants per protocol condition per case cluster.

3. CASE STRATIFICATION: difficulty (3) × model_correctness (2) = 6 cells × 3 cases = 18 cases

WHY THESE CELLS: - easy+correct → baseline, participant should agree with AI - easy+wrong → automation bias check (AI is confident but wrong) - medium+correct → moderate uncertainty, correct AI - medium+wrong → medium uncertainty, wrong AI — normative gap is largest here - hard+correct → under-reliance check (AI uncertain but correct) - hard+wrong → highest ambiguity, wrong AI

4. Confidence is recorded as a covariate, not a design factor. This preserves statistical power while maintaining interpretability.

Stratified case selection

```
[10]: N_PER_CELL = 3

df_pool = df[
    (df['pred_prob'] > 0.05) &
    (df['pred_prob'] < 0.95) &
    (df['difficulty_tier'].notna())
].copy()

df_pool = df_pool[df_pool['purpose'] != 'other'].copy()

print(f"Case pool after filtering: {len(df_pool):,}")
print(df_pool.groupby(['difficulty_tier', 'correct'], observed=True).size().
      ↪unstack(fill_value=0))

def sample_cell(df, tier, correct, n):
    subset = df[
        (df['difficulty_tier'] == tier) &
        (df['correct'] == correct)
    ].copy()

    if len(subset) < n:
        return subset
```

```

    if 'model_optimal' in subset.columns and subset['model_optimal'].nunique() > 1:
        subset = subset.sort_values(['model_optimal', 'pred_prob'])
    else:
        subset = subset.sort_values('pred_prob')

    q = min(3, n)
    subset['prob_bin'] = pd.qcut(subset['pred_prob'], q=q, duplicates='drop')

    selected = []
    used_purposes = set()

    for _, group in subset.groupby(['prob_bin', 'model_optimal'], dropna=False):
        group = group.sample(frac=1, random_state=42)
        for _, row in group.iterrows():
            if row['purpose'] not in used_purposes:
                selected.append(row)
                used_purposes.add(row['purpose'])
                break
        if len(selected) >= n:
            break

    if len(selected) < n:
        remaining = subset.drop(index=[r.name for r in selected])
        remaining = remaining.sample(frac=1, random_state=42)
        for _, row in remaining.iterrows():
            selected.append(row)
            if len(selected) == n:
                break

    return pd.DataFrame(selected)

cells = [
    ('easy', 1),
    ('easy', 0),
    ('medium', 1),
    ('medium', 0),
    ('hard', 1),
    ('hard', 0),
]

sampled = [
    sample_cell(df_pool, tier, correct, N_PER_CELL)
    for tier, correct in cells
]

```

```

cases = pd.concat(sampled).drop_duplicates().reset_index(drop=True)
cases['case_position'] = cases.index + 1

print(f"\nSELECTED: {len(cases)}")

```

Case pool after filtering: 417,304

correct	0	1
difficulty_tier		
easy	29621	168249
hard	24215	34277
medium	48155	112787

SELECTED: 18

Protocol assignment

```

[11]: # Assign each case to a protocol block.
      # Three blocks of 6 cases each.
      # Counterbalancing handled at the participant level (in the app).

cases = cases.sort_values(['difficulty_tier', 'correct', 'pred_prob']).
      ↪reset_index(drop=True)
cases['case_position'] = cases.index + 1

# Assign cases to blocks (6 cases per block)
cases['block'] = pd.cut(
    cases.index,
    bins=[-1, 5, 11, 17],
    labels=['block_1', 'block_2', 'block_3']
)

# Protocol rotation table (shown in app, not hardcoded per case)
protocol_rotation = pd.DataFrame({
    'participant_group': ['group_1', 'group_2', 'group_3'],
    'block_1_protocol' : ['no_ai',      'human_first', 'ai_first'],
    'block_2_protocol' : ['human_first', 'ai_first',   'no_ai'],
    'block_3_protocol' : ['ai_first',   'no_ai',      'human_first'],
    'target_n'         : [33, 33, 34]
})

print("=== PROTOCOL ROTATION TABLE ===\n")
print(protocol_rotation.to_string(index=False))
print("\nEach participant is assigned to one group.")
print("Within their group, every case is seen under exactly one protocol.")
print("Across 100 participants, every case appears under every protocol.\n")

```

=== PROTOCOL ROTATION TABLE ===

participant_group	block_1_protocol	block_2_protocol	block_3_protocol	target_n
-------------------	------------------	------------------	------------------	----------

group_1	no_ai	human_first	ai_first	33
group_2	human_first	ai_first	no_ai	33
group_3	ai_first	no_ai	human_first	34

Each participant is assigned to one group.
 Within their group, every case is seen under exactly one protocol.
 Across 100 participants, every case appears under every protocol.

Distribution checks

```
[12]: print("=== CASE DISTRIBUTION ===\n")

print("By difficulty:")
print(cases['difficulty_tier'].value_counts().sort_index())

print("\nBy correctness:")
print(cases['correct'].value_counts())

print("\nBy block:")
print(cases['block'].value_counts().sort_index())

print("\nCross-tab (difficulty × correctness):")
print(pd.crosstab(cases['difficulty_tier'], cases['correct']))

print("\nPredicted probability by cell:")
print(
    cases.groupby(
        ['difficulty_tier', 'correct'], observed=True
    )['pred_prob'].agg(['min', 'mean', 'max']).round(3)
)

print("\nConfidence by cell:")
print(
    cases.groupby(
        ['difficulty_tier', 'correct'], observed=True
    )['confidence'].agg(['min', 'mean', 'max']).round(3)
)
```

```
=== CASE DISTRIBUTION ===
```

```
By difficulty:
difficulty_tier
easy          6
hard          6
medium        6
Name: count, dtype: int64
```

```
By correctness:
```

```
correct
0      9
1      9
Name: count, dtype: int64
```

```
By block:
block
block_1      6
block_2      6
block_3      6
Name: count, dtype: int64
```

```
Cross-tab (difficulty × correctness):
correct      0  1
difficulty_tier
easy          3  3
hard          3  3
medium        3  3
```

```
Predicted probability by cell:
              min    mean    max
difficulty_tier correct
easy          0      0.114  0.142  0.169
              1      0.055  0.134  0.234
hard          0      0.145  0.305  0.423
              1      0.114  0.254  0.397
medium        0      0.114  0.200  0.251
              1      0.145  0.192  0.251
```

```
Confidence by cell:
              min    mean    max
difficulty_tier correct
easy          0      0.831  0.858  0.886
              1      0.766  0.866  0.945
hard          0      0.577  0.695  0.855
              1      0.603  0.746  0.886
medium        0      0.749  0.800  0.886
              1      0.749  0.808  0.855
```

Probability distribution

```
[13]: print("\n=== PROBABILITY SUMMARY ===\n")
      print(cases['pred_prob'].describe())
```

```
=== PROBABILITY SUMMARY ===
```

```
count      18.000000
mean        0.204631
```

```

std      0.103099
min      0.054726
25%      0.121801
50%      0.175288
75%      0.250760
max      0.422535
Name: pred_prob, dtype: float64

```

Difficulty x correctness interaction

```

[14]: print("=== DIFFICULTY × CORRECTNESS INTERACTION (full test set) ===\n")
      interaction = (
          df.groupby(['difficulty_tier', 'correct'], observed=True)
            .agg(
                n          = ('pred_prob', 'size'),
                avg_pred    = ('pred_prob', 'mean'),
                avg_conf    = ('confidence', 'mean'),
                model_opt   = ('model_optimal', 'mean')
            )
            .round(3)
        )
      print(interaction.to_string())

      fig, axes = plt.subplots(1, 2, figsize=(12, 4))

      # Accuracy
      sns.barplot(
          data=df, x='difficulty_tier', y='correct',
          hue='difficulty_tier', palette=['steelblue', 'orange', 'tomato'],
          ax=axes[0], legend=False, order=['easy', 'medium', 'hard']
      )
      axes[0].set_title("Model Accuracy by Difficulty")
      axes[0].set_ylabel("Accuracy Rate")
      axes[0].set_ylim(0, 1)

      # Normative agreement
      sns.barplot(
          data=df, x='difficulty_tier', y='model_optimal',
          hue='difficulty_tier', palette=['steelblue', 'orange', 'tomato'],
          ax=axes[1], legend=False, order=['easy', 'medium', 'hard']
      )
      axes[1].set_title("Model-Optimal Agreement by Difficulty")
      axes[1].set_ylabel("Agreement Rate")
      axes[1].set_ylim(0, 1)

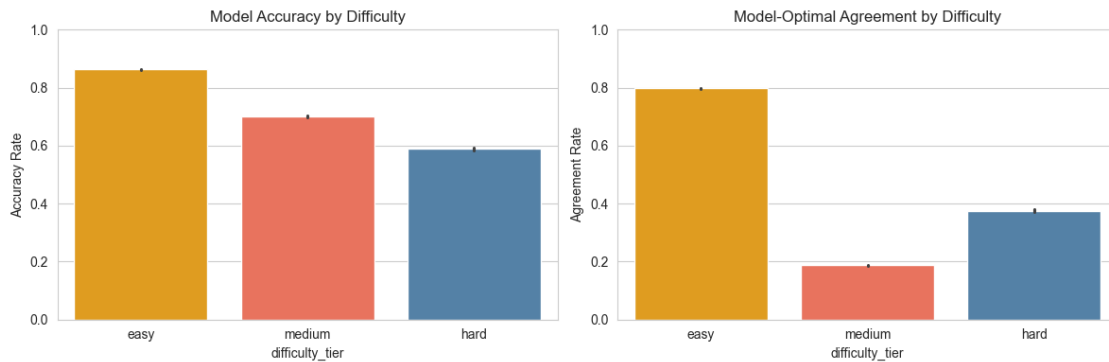
      plt.tight_layout()
      plt.savefig("../visualizations/difficulty_interactions.png", dpi=150,
                  bbox_inches='tight')

```

```
plt.show()
```

=== DIFFICULTY × CORRECTNESS INTERACTION (full test set) ===

		n	avg_pred	avg_conf	model_opt
easy	0	33044	0.137	0.863	0.672
	1	208037	0.108	0.892	0.819
hard	0	26000	0.459	0.615	0.462
	1	37224	0.419	0.638	0.316
medium	0	52384	0.312	0.706	0.210
	1	123055	0.262	0.745	0.177



Final export

```
[15]: os.makedirs("../..//artifacts", exist_ok=True)

EXPORT_COLS = [
    'case_id', 'case_position', 'block',
    'loan_amnt', 'term', 'int_rate', 'log_annual_inc',
    'dti', 'revol_util', 'home_ownership', 'purpose',
    'credit_history_years',
    'pred_prob', 'y_true', 'difficulty_tier', 'difficulty_score',
    'correct', 'confidence', 'conf_bin', 'model_optimal'
]

final_cases = cases[[c for c in EXPORT_COLS if c in cases.columns]].copy()

final_cases.to_csv("../..//artifacts/final_cases.csv", index=False)
protocol_rotation.to_csv("../..//artifacts/protocol_rotation.csv", index=False)

print("Saved:")
print("  ../..//artifacts/final_cases.csv")
print("  ../..//artifacts/protocol_rotation.csv")
```

```
print(f"\nColumns in final_cases: {list(final_cases.columns)}")
print(f"Total cases: {len(final_cases)}")
print(f"Trials per participant: {len(final_cases)}")
```

Saved:

```
../..artifacts/final_cases.csv
../..artifacts/protocol_rotation.csv
```

Columns in final_cases: ['case_id', 'case_position', 'block', 'loan_amnt', 'term', 'int_rate', 'log_annual_inc', 'dti', 'revol_util', 'home_ownership', 'purpose', 'credit_history_years', 'pred_prob', 'y_true', 'difficulty_tier', 'difficulty_score', 'correct', 'confidence', 'conf_bin', 'model_optimal']

Total cases: 18

Trials per participant: 18

Feature interpretation helpers

```
[16]: def dti_label(dti):
        if dti < 15: return "LOW"
        elif dti < 30: return "MODERATE"
        else: return "HIGH"

    def util_label(x):
        if x < 30: return "LOW"
        elif x < 70: return "MODERATE"
        else: return "HIGH"

    def income_label(x):
        if x < 30000: return "LOW"
        elif x < 100000: return "MEDIUM"
        else: return "HIGH"
```

Full case preview

INSTRUCTIONS

You are acting as a loan officer.

If you approve a loan and the borrower defaults → LARGE LOSS If you reject a good loan → SMALL LOSS

Your goal is to make decisions that minimize total loss.

Your performance will be evaluated based on decision quality.

```
[17]: def render_case(row, protocol='ai_first', show_index=True):
        income = int(np.exp(row['log_annual_inc']))
        hist = round(row['credit_history_years'], 1)
        pred_pct = int(round(row['pred_prob'] * 100))
        tier = str(row['difficulty_tier']).upper()
```

```

# labels
dti_l = dti_label(row['dti'])
util_l = util_label(row['revol_util'])
inc_l = income_label(income)

header = ""
if show_index:
    header = (
        f"\n{'='*60}\n"
        f"CASE {int(row['case_position']):02d} of 18 | Block:␣
↪{row['block']} | [{tier}]\n"
        f"{'='*60}"
    )

profile = (
    f"\nLOAN APPLICATION\n"
    f"{'-'*44}\n"
    f"Loan amount      : ${row['loan_amnt']:>10,.0f}\n"
    f"Interest rate     : {row['int_rate']:>9.2f}%\n"
    f"Annual income      : ${income:>10,.0f} ({inc_l})\n"
    f"Debt-to-income     : {row['dti']:>9.1f} ({dti_l})\n"
    f"Revolving util.    : {row['revol_util']:>9.1f}% ({util_l})\n"
    f"Credit history     : {hist:>7.1f} years\n"
    f"Home ownership     : {str(row['home_ownership']):>12}\n"
    f"Loan purpose       : {str(row['purpose']):>12}\n"
    f"Term               : {str(row['term']):>12}\n"
    f"{'-'*44}"
)

# -----
# PROTOCOL LOGIC
# -----

if protocol == 'no_ai':
    decision_block = (
        f"\n\nDECISION TASK:\n"
        f"Would you APPROVE or REJECT this loan?\n"
        f"[Approve] [Reject]\n\n"
        f"(Optional) What probability do you assign to default?\n"
        f"[ 0% |-----| 100% ]\n\n"
        f"How confident are you? (0-100%)\n"
        f"[ 0% |-----| 100% ]\n"
    )

elif protocol == 'human_first':
    decision_block = (
        f"\n\nSTEP 1 - Initial decision:\n"

```

```

        f"Would you APPROVE or REJECT?\n"
        f"[Approve] [Reject]\n\n"
        f"(Optional) Initial probability:\n"
        f"[ 0% |-----| 100% ]\n"
        f"[SUBMIT]\n\n"
        f"--- AI revealed ---\n\n"
        f"AI ASSESSMENT:\n"
        f"Default probability: {pred_pct}%\n\n"
        f"STEP 2 - Final decision:\n"
        f"[Approve] [Reject]\n\n"
        f"(Optional) Final probability:\n"
        f"[ 0% |-----| 100% ]\n\n"
        f"Confidence (0-100%):\n"
        f"[ 0% |-----| 100% ]\n"
    )

else: # ai_first
    decision_block = (
        f"\n\nAI ASSESSMENT:\n"
        f"Default probability: {pred_pct}%\n\n"
        f"DECISION TASK:\n"
        f"Would you APPROVE or REJECT?\n"
        f"[Approve] [Reject]\n\n"
        f"(Optional) Probability estimate:\n"
        f"[ 0% |-----| 100% ]\n\n"
        f"Confidence (0-100%):\n"
        f"[ 0% |-----| 100% ]\n"
    )

footer = f"\n{'-'*44}\n[SUBMIT]\n{'='*60}\n"

print(header + profile + decision_block + footer)

```

Summary table

```

[18]: summary = final_cases[[
        'case_position', 'block', 'difficulty_tier',
        'correct', 'pred_prob', 'confidence',
        'y_true', 'model_optimal',
        'home_ownership', 'purpose', 'loan_amnt', 'int_rate'
    ]].copy()

summary.columns = [
    'pos', 'block', 'difficulty',
    'model_correct', 'ai_pred', 'ai_conf',
    'true_label', 'normative_align',
    'ownership', 'purpose', 'loan_amnt', 'int_rate'

```

```

]

summary['ai_pred'] = summary['ai_pred'].round(3)
summary['ai_conf'] = summary['ai_conf'].round(3)

print("=== ALL 18 EXPERIMENTAL CASES ===\n")
print(summary.to_string(index=False))

print("\n=== KEY STATISTICS ===")
print(f"Cases where model is correct : "
      f"{int(summary['model_correct'].sum())} / 18")
print(f"Cases aligned with optimal : "
      f"{int(summary['normative_align'].sum())} / 18")
print(f"True default rate in cases : "
      f"{summary['true_label'].mean():.3f}")
print(f"Mean AI prediction : "
      f"{summary['ai_pred'].mean():.3f}")
print(f"Mean AI confidence : "
      f"{summary['ai_conf'].mean():.3f}")
print(f"Prediction range : "
      f"{summary['ai_pred'].min():.3f}, {summary['ai_pred'].max():.3f}")

print("\n=== DESIGN BALANCE CHECK ===")
print(pd.crosstab(
    summary['difficulty'],
    [summary['model_correct'], summary['normative_align']],
    rownames=['difficulty'],
    colnames=['model_correct', 'normative_align']
))

```

=== ALL 18 EXPERIMENTAL CASES ===

	pos	block	difficulty	model_correct	ai_pred	ai_conf	true_label
		normative_align	ownership		purpose	loan_amnt	int_rate
	1	block_1	easy	0	0.114	0.886	1
1		RENT	debt_consolidation	18500	9.16		
	2	block_1	easy	0	0.145	0.855	1
1		MORTGAGE	house	8000	12.62		
	3	block_1	easy	0	0.169	0.831	1
0		RENT	credit_card	4000	9.44		
	4	block_1	easy	1	0.055	0.945	0
1		MORTGAGE	debt_consolidation	19000	7.99		
	5	block_1	easy	1	0.114	0.886	0
1		MORTGAGE	credit_card	4850	11.06		
	6	block_1	easy	1	0.234	0.766	0
0		MORTGAGE	major_purchase	40000	12.13		
	7	block_2	hard	0	0.145	0.855	1
1		MORTGAGE	house	25000	16.02		

8	block_2	hard	0	0.347	0.653	1
0	MORTGAGE	debt_consolidation	6000	16.99		
9	block_2	hard	0	0.423	0.577	1
0	RENT	credit_card	10000	19.99		
10	block_2	hard	1	0.114	0.886	0
1	MORTGAGE	major_purchase	9600	16.02		
11	block_2	hard	1	0.251	0.749	0
0	RENT	debt_consolidation	6675	15.99		
12	block_2	hard	1	0.397	0.603	0
0	MORTGAGE	credit_card	16000	18.45		
13	block_3	medium	0	0.114	0.886	1
1	MORTGAGE	home_improvement	4000	11.44		
14	block_3	medium	0	0.234	0.766	1
0	RENT	debt_consolidation	4000	11.49		
15	block_3	medium	0	0.251	0.749	1
0	RENT	credit_card	15000	14.46		
16	block_3	medium	1	0.145	0.855	0
1	MORTGAGE	credit_card	6000	13.49		
17	block_3	medium	1	0.182	0.818	0
0	MORTGAGE	debt_consolidation	4800	10.99		
18	block_3	medium	1	0.251	0.749	0
0	MORTGAGE	home_improvement	6000	19.99		

=== KEY STATISTICS ===

Cases where model is correct : 9 / 18
Cases aligned with optimal : 8 / 18
True default rate in cases : 0.500
Mean AI prediction : 0.205
Mean AI confidence : 0.795
Prediction range : [0.055, 0.423]

=== DESIGN BALANCE CHECK ===

model_correct	0	1		
normative_align	0	1	0	1
difficulty				
easy	1	2	1	2
hard	2	1	2	1
medium	2	1	2	1